

# UNIVERSAL UNIT CONVERTER

## Internship Project Report

**Submitted by:** Akhinesh Nair

**Intern ID:** CITS6205

**Organization:** CodTech IT Solutions

## Acknowledgement

Thank you CodTech IT Solutions for providing me the opportunity for completing this internship project. This project allowed me to gain more knowledge in python programming. Through this project I understood basics of web development using Flask framework, how to design web pages using HTML5, CSS for styling the web pages, and javascript for responsive web pages and input validation, how to connect the frontend with the backend using Flask and other important topics.

## Introduction

The Universal Unit Converter is a web-based application developed using Python, Flask, HTML, CSS, and JavaScript. It is designed to provide a simple, fast, and accurate way to convert values between different units of measurement through an easy-to-use interface.

The application supports conversions for Length, Weight, Temperature, Speed, Time, Area, and Volume. Users simply select a category, choose the "From" and "To" units, enter a value, and the application instantly displays the converted result. It also includes features such as dynamic unit selection, input validation, a swap units button, and an About page, making it user-friendly and efficient.

This project demonstrates the integration of frontend and backend technologies using Flask. It also enhances practical knowledge of Python programming, web development, and responsive user interface design while providing a useful tool for everyday unit conversions.

## Objectives

- Develop a responsive web application.
- Perform accurate unit conversions.
- Provide a simple and attractive user interface.
- Improve programming skills using flask.
- Demonstrate web development concepts.

## Technologies used

| Technology  | Purpose                |
|-------------|------------------------|
| Python 3.12 | Programming Language.  |
| Flask       | Web Framework.         |
| HTML5       | Web Page Structure.    |
| CSS3        | Styling.               |
| Javascript  | Dynamic Functionality. |
| VS Code     | IDE.                   |
| Github      | Version Control        |

## Project Structure

```
Unit-Converter/  
    app.py  
    converter.py  
    README.md  
    requirements.txt  
    templates/  
        index.html  
        about.html  
    static/  
        style.css  
        script.js  
    screenshots/  
    documentation/
```

### File description:

- **app.py** – Main Flask app that handles routes, user, requests, and display the web pages.
- **converter.py** – Contains the conversion logic and formulas.
- **templates/index.html** – Home page providing user interface for performing conversions.

- **templates/about.html** – Displays information about the project.
- **static/style.css** – Defines the layout, colors and responsive styling of the web application.
- **static/script.js** – Implements dynamic dropdowns, input validation, and the swap button feature.
- **requirements.txt** – Lists the required Python packages.
- **README.md** – Contains project information and instructions.
- **screenshots/** – Stores screenshots of the application's output.
- **documentation/** – Contains the project report and related documents.
- **gitignore** – Specifies files and folders that git should ignore whrn uploading the project.

## Working

### ❖ Application Startup

- The Flask server starts by running app.py.
- The home page (index.html) is loaded in the user's web browser.

### ❖ Category Selection

- The user selects a conversion category such as Length, Weight, Temperature, Speed, Time, Area, or Volume.
- JavaScript dynamically updates the From and To dropdown menus with the relevant units for the selected category.

### ❖ User Input

- The user enters a numerical value to be converted.
- The user selects the source unit (From) and the destination unit (To).

### ❖ Input Validation

- JavaScript checks that the input field is not empty.
- It verifies that the entered value is a valid number.
- It ensures that the source and destination units are different before allowing the conversion.

### ❖ **Conversion Process**

- When the Convert button is clicked, the form data is sent to the Flask application.
- The app.py file receives the user input and passes it to the converter.py module.
- The appropriate conversion formula is applied based on the selected category and units.

### ❖ **Displaying the Result**

- The converted value is returned to app.py.
- Flask sends the result back to index.html.
- The converted value is displayed clearly on the webpage.

### ❖ **Swap Units Feature**

- The Swap button allows users to exchange the selected From and To units with a single click.
- This makes reverse conversions faster and more convenient.



### ❖ **Reset Function**

- The Reset button clears the entered value and restores the form to its default state, allowing the user to start a new conversion.

### ❖ **About Page**

- The About page provides information about the application, its features, and the technologies used to develop it.

### ❖ **Responsive Design**

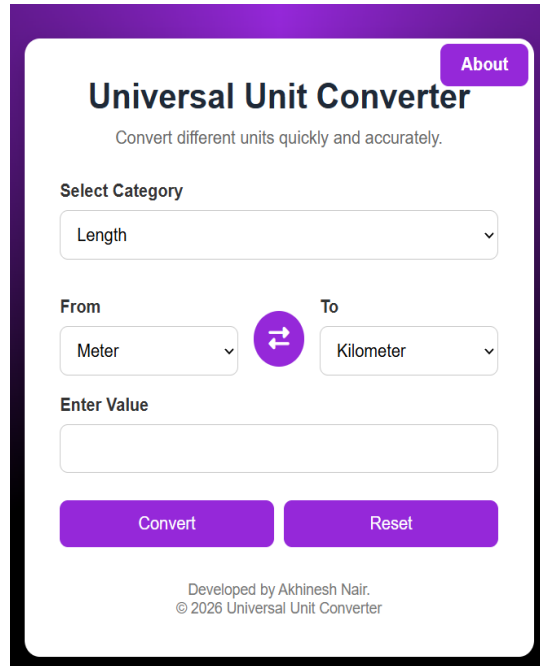
- The application is designed to work smoothly on both desktop and mobile devices, ensuring a consistent user experience across different screen sizes.

## Features

- I. Length converter
- II. Weight converter
- III. Temperature converter
- IV. Speed converter
- V. Time converter
- VI. Area converter
- VII. Volume converter
- VIII. Dynamic dropdowns
- IX. Swap unit feature
- X. Input validation
- XI. Responsive design
- XII. About page

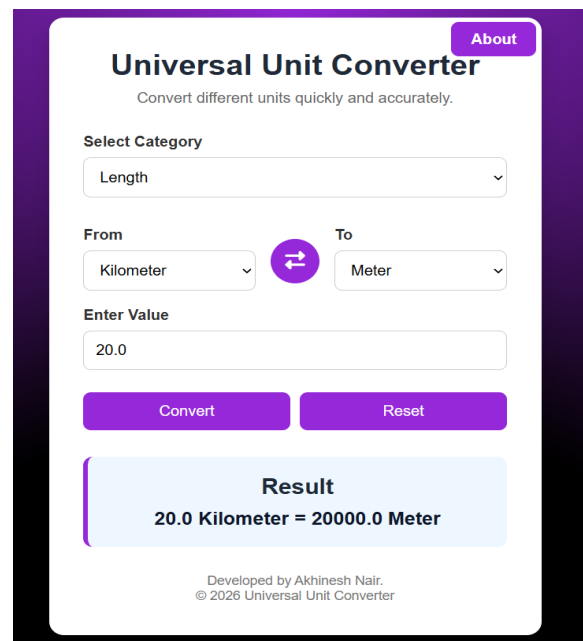
## Screenshots

Home page:-



The screenshot shows the home page of the Universal Unit Converter. It features a purple header with the title "Universal Unit Converter" and a subtitle "Convert different units quickly and accurately." Below the title is a "Select Category" dropdown menu set to "Length". Underneath, there are two dropdown menus for "From" (set to "Meter") and "To" (set to "Kilometer"), separated by a circular swap button. A text input field labeled "Enter Value" is empty. At the bottom, there are two buttons: "Convert" and "Reset". The footer includes the text "Developed by Akhinesh Nair. © 2026 Universal Unit Converter" and an "About" link in the top right corner.

Length conversion:-



This screenshot shows the Universal Unit Converter with a length conversion. The "From" unit is set to "Kilometer" and the "To" unit is set to "Meter". The "Enter Value" field contains "20.0". The "Convert" button has been clicked, and the result is displayed in a light blue box: "20.0 Kilometer = 20000.0 Meter". The rest of the interface, including the header, category dropdown, and footer, remains the same as in the home page screenshot.

weight conversion:-

The screenshot displays the 'Universal Unit Converter' web application. At the top right is an 'About' button. The main heading is 'Universal Unit Converter' with a subtitle 'Convert different units quickly and accurately.' Below this is a 'Select Category' dropdown menu set to 'Weight'. Underneath are 'From' and 'To' unit selection dropdowns, currently showing 'Kilogram' and 'Gram' respectively, with a swap icon between them. An 'Enter Value' input field contains '23.0'. Below the input are 'Convert' and 'Reset' buttons. A light blue box displays the 'Result' as '23.0 Kilogram = 23000.0 Gram'. At the bottom, it says 'Developed by Akhinesh Nair. © 2026 Universal Unit Converter'.

about page:-

The screenshot shows the 'About' page of the web application. It features a heading 'About' followed by a paragraph: 'The Universal Unit Converter is a web application that allows users to convert values between different measurement units quickly and accurately.' Below this are two sections: 'Features' and 'Technologies Used'. The 'Features' section lists: Length Conversion, Weight Conversion, Temperature Conversion, Speed Conversion, Time Conversion, Area Conversion, Volume Conversion, Swap Units, and Responsive Design. The 'Technologies Used' section lists: Python, Flask, HTML5, CSS3, and JavaScript. At the bottom is a button labeled '← Back to Converter'.

Before Swapping from and to units:-

The screenshot shows the 'Universal Unit Converter' interface. At the top right is an 'About' button. The title 'Universal Unit Converter' is centered, with the subtitle 'Convert different units quickly and accurately.' below it. Under 'Select Category', a dropdown menu shows 'Speed'. The 'From' unit is 'km/h' and the 'To' unit is 'm/s', with a swap icon between them. The 'Enter Value' field contains '23.0'. Below the input field are 'Convert' and 'Reset' buttons. The 'Result' box displays '23.0 km/h = 6.3889 m/s'. At the bottom, it says 'Developed by Akhinesh Nair. © 2026 Universal Unit Converter'.

After Swapping:-

The screenshot shows the 'Universal Unit Converter' interface after swapping the units. The 'From' unit is now 'm/s' and the 'To' unit is 'km/h'. The 'Enter Value' field still contains '23.0'. The 'Result' box now displays '23.0 m/s = 82.7999 km/h'. All other elements, including the 'About' button, category dropdown, and footer, remain the same.

## Testing

| Test case         | Expected Result    | Status |
|-------------------|--------------------|--------|
| Valid Input       | Correct conversion | Pass   |
| Invalid Input     | Error Message      | Pass   |
| Swap Button       | Unit Swapped       | Pass   |
| Responsive Design | Layout Adjusts     | Pass   |
| Category Change   | Units Updated      | Pass   |

## Future Enhancements

- Currency converter
- Conversion History
- Copy Result Button
- More Unit categories
- Multiple Language Support
- Voice Input
- Scientific Calculator

## Conclusion

The Universal Unit Converter was successfully developed using Python, Flask, HTML, CSS, and JavaScript. The application provides a simple, responsive, and user-friendly interface for converting values across multiple categories, including Length, Weight, Temperature, Speed, Time, Area, and Volume. Features such as dynamic unit selection, input validation, the swap function, and an About page enhance the overall user experience and make the application practical for everyday use.

Developing this project provided valuable hands-on experience in both frontend and backend web development. It strengthened my understanding of Flask, JavaScript, responsive web design, and project organization while improving my problem-solving and debugging skills. Overall, this project demonstrates the successful integration of multiple web technologies to create a reliable and efficient application.

## References

- [1] Python documentation
- [2] Javascript documentation
- [3] HTML5 documentation
- [4] Flask Documentation
- [5] ChatGPT
- [6] Font Awesome